Workout RAG Assistant

Requirements and Design | Baseline, October 2026 | Prajna Shetty

Part 1 says what the system must do and where its boundaries are. Part 2 says how it is built: the architecture, the technology, the folder structure and the build order.

Part 1: Requirements

1. Problem Statement

A chat assistant answers questions about the user's own workout plan and history using only retrieved data, never the language model's own knowledge. For exercise form and for suggestions outside the plan it uses web sources, clearly labeled. When the evidence is not sufficient, the assistant asks the user or declines to recommend. Every recommendation can be explained by pointing to the evidence behind it.

2. Scope

The assistant will:

- Tell the user today's or tomorrow's workout.
- Recommend a weight for an exercise from the last recorded performance and a progression rule.
- Compare workout history, for example this week against last week.
- Explain any recommendation it made earlier in the same chat.
- Ask a clarifying question when needed, then continue from where it paused.
- Answer exercise form and technique questions, and suggest exercises that are not in the current plan, from labeled web sources.
- Remember the conversation within one chat.

The assistant will not (in this version):

- Log or edit workouts. It only reads data.
- Remember anything across chats, or keep a user profile.
- Write back to Jefit or any other service.
- Diagnose, or give rehabilitation or treatment advice.
- Support several users. There is one user; separate chats are still isolated from each other.
- Use web content in any calculation, or present it as the user's own data.

Context. One user, working with her own Jefit export on a MacBook (M3, 16 GB). The system runs on the laptop first, then in containers, and later reachable from a phone.

3. What the System Must Do

Choosing the workout day

- The plan has numbered days, not dates. The user trains Monday to Saturday in the order Push A, Pull A, Legs A, Push B, Pull B, Leg B, and the sequence shifts when a day is skipped. Arms is not part of the rotation and is left out.
- For "today" or "tomorrow" the assistant proposes the least recently done of the six rotation days, says that this is its assumption, and asks the user to confirm. The weekday (Push B is usually Thursday) is only a tie-break.
- Which day a past session was is inferred from the exercises actually logged, matched against each plan day. If the match is unclear, the assistant asks.

Recommending a weight

- A recommendation needs the last weight, the last reps, the target reps and an applicable rule. If any is missing, the assistant asks.

- When all target reps were completed on every set (targets compared set by set), the next weight is the last weight plus 2.5 kg for compound lifts, or plus 1 kg for isolation lifts.
- For assisted machines, such as the assisted pull-up, progress means less assistance: the assistance weight goes down in 5 kg steps.
- If targets are missed, the weight stays the same, and is reduced after repeated misses.
- Every exercise carries a tag: compound, isolation or assisted. The language model suggests tags and the user reviews and approves them. An untagged exercise gets no recommendation; the assistant asks.

Retrieval and evidence

- Questions can need several lookups where the next depends on the previous (multi-hop). Independent lookups run at the same time.
- Before answering, the assistant checks whether the evidence is sufficient, missing or conflicting.
- When something is missing or ambiguous, for example "squat" matching several exercises, it asks the user, remembers what it was asking, and resumes from that point when the user replies, without starting over.
- Every recommendation carries an evidence record (source rows and the rule applied), so "Why?" is answered from stored evidence.
- Structured data extracted from free text is validated against a schema. Invalid output never enters any store.

Sessions

- Concurrent chats are isolated: nothing from one chat can appear in another.
- Chat history is capped at 20 user turns. When the cap is reached, the assistant says so and offers a new chat. Saved state such as the last recommendation is kept separately and is not lost by the cap.

4. Data

Data	Source	Notes
Workout history	Jefit export: exercise logs (1,299 logs over 150 days, 171 exercises, Aug 2022 to Oct 2026) and the per-set table	Sets are stored as text such as 70x10,70x11,70x12.
Weekly plan	Jefit export: the routine "PPL" with 7 days and 98 planned exercises (53 distinct), with set counts and per-set target reps	Days are numbered, not dated.
Personal records	Jefit export: 157 records	A possible input to progression.
Progression rules	A rules file written by the user	Not available from Jefit.
Form and technique, out-of-plan exercises	Web pages: ACE Fitness exercise library and Jefit exercise pages	Jefit has only 4 notes, none useful.
Conversation state	Session store	Short-term only.

Storage principle. The Jefit CSV is imported into SQL. SQL rows are the only source of facts such as weights, reps, dates and the plan. Embeddings are used only to find things by meaning, for example matching the user's words to an exercise name. Numbers are never read from embedded text. The reasons: questions like "last squat" or "maximum last week" need exact ordering and aggregation; near-identical numbers (70x10 and 60x10) embed almost identically, so similarity cannot be trusted for digits; evidence and explanations need row identifiers; conflict detection needs keys; and cleaning is done once at import.

What the Jefit export looks like

Finding	Consequence	How it is handled
One CSV holds about a dozen tables under "### SECTION ###" markers, and headers repeat.	A plain CSV reader fails.	The importer splits by section.
Plan days have numbers (Push A, Pull A, Legs A, Arms, Push B, Pull B, Leg B) but no weekday or date.	"Tomorrow" cannot be read from the file.	Least recently done rotation day, confirmed with the user.

Finding	Consequence	How it is handled
The completion timestamps in the days table are unreliable. Push A shows the export time.	They cannot tell which day was last done.	Infer from logged exercises.
Matching a session's exercises to plan days works but is often unclear: in 2026, 13 of 31 sessions match clearly.	Day inference is sometimes uncertain.	Ask when unclear; the weekday is a tie-break.
Planned exercises join to days on belongplan = days._id. The day numbering is the display order, not the key.	Using day numbers would pair exercises with the wrong day.	Join on the identifier. Confirmed: Thursday sessions match Push B.
Weights before 2026 are in pounds and from 2026 are in kilograms. Median top weights for the same exercises are 2 to 3 times lower in 2026.	Mixed units would corrupt comparisons.	Convert logs dated before 2026-01-01 to kg; keep the raw value.
There are no logs between 2025-11-11 (two entries) and 2026-08-26. The kilogram era has about six weeks of history (249 logs).	The unit change cannot be seen as a drop; recent history is short.	The 2026-01-01 cutoff gives the same result as any date in the gap.
34 of 1,301 logs contain values like 34.9999.	Rounding artifacts.	Round to 2 decimals at import.
The logs text and the per-set table describe the same sets.	A built-in chance of conflict.	Detect and treat as a conflict.
"Squat" matches Hack Squat (Reverse Position), split squats and others.	Ambiguous exercise names.	Ask the user.
The file contains the date of birth and body measurements.	Privacy.	Keep the data folder out of version control.

Freshness. The export is a snapshot. The assistant states the date range of the data it used and asks about anything newer. Values the user gives during a clarification are held for that chat only; they are not written to the history store or to Jefit.

5. How the Assistant Decides

Outcome	When	Who can resolve it
Retrieve	Any question that depends on the user's plan, history or rules (from SQL); form, technique and out-of-plan questions (from the web, labeled).	The system
No retrieval	Small talk, or a follow-up that stored evidence already answers ("Why?").	Not applicable
Ask	A required value is missing and the user could plausibly know it; a reference, exercise name or date range is ambiguous; records conflict (always ask, even when the user's value differs from the data); the answer depends on how the user feels now; a needed input such as the one-rep max is missing; the user returns to an earlier topic.	The user
Refuse	Acute pain or injury; requests for diagnosis or rehabilitation; unsafe or extreme requests; evidence still insufficient after the user was asked.	Nobody
System failure	A database, vector store, language model, session store or web call times out or errors. The assistant says so. It does not guess, and does not ask the user for data as a workaround.	Retry later

6. Short-Term Memory

Aspect	Behaviour
Remembered	Chat messages (up to the cap) and structured state: the last recommendation with its evidence, any pending question, and values the user supplied in this chat.
Forgotten	When the session ends.

Aspect	Behaviour
Not remembered	Anything from other chats.
Returning to a topic	All topics in a chat are kept. When the user returns to an earlier topic, the assistant always asks the user to confirm which topic is meant.
Session end	An idle timeout or an explicit "new chat". A timeout does not discard a pending clarification question.
Long chats	A cap of 20 user turns. No summarization, because a summary written by a small model could silently change a weight.

7. What the Assistant Must Never Do

- State a workout fact (weight, reps, date, plan) that was not retrieved from the user's data or given by the user.
- Invent a weight.
- Claim a lookup succeeded when it failed or returned nothing.
- Diagnose, or give rehabilitation or treatment advice.
- Answer from the language model's own knowledge when the required data or web source is unavailable.
- Present web content as the user's data, or use it in a weight calculation.

8. Web Lookup Rules

- Every web-sourced statement is labeled as such and shows its source address.
- Web content is used only for form and technique, and for suggestions outside the plan. Those suggestions come from exercises already in the user's Jefit catalog (the 171 logged exercises).
- The page address is built from the exercise name (direct fetch of known pages). There is no search service.
- If the lookup fails or times out, the assistant says so. It does not fall back to the model's own knowledge.
- Page text is treated as data, never as instructions.
- Safety refusals still apply to web-based answers.
- Approved sources: the ACE Fitness exercise library and Jefit exercise pages. Both were readable as text and their robots files allow it; their terms of use have not been reviewed. Keep use low-volume, and store only short summaries with a link back. YouTube is postponed until the user chooses channels.

9. Non-Functional Requirements

- All input and output (language model, database, vector search, web, session store) is asynchronous. Nothing blocks the event loop.
- Independent retrievals run concurrently; dependent ones run in order.
- Many chats can run at once without sharing state, and there is no unsafe global mutable state.
- These failures are handled and tested: language model timeout, database timeout, vector store failure, web failure, malformed model output, empty retrieval, conflicting records, simultaneous users, a clarification reply, and a change of subject.
- The language model runs locally. Personal data stays on the machine and out of version control.
- Every recommendation can be explained from stored evidence.

10. User Journeys

Journey	The user says	It ends with	What it tests
Simple retrieval	"Did I lift more on bench last week?"	An answer that states whether last week means the calendar week or the last 7 days	One lookup; date ambiguity

Journey	The user says	It ends with	What it tests
Weight for tomorrow	"What weight for squats tomorrow?"	A question about which squat, then an answer with evidence	Several lookups; name matching; concurrent lookups
Missing data	The same, with no last performance on record	A question, the user's reply, then an answer	Missing data; resuming without restarting
Conflict	The user states a weight that differs from the data	A question naming both values	Conflict handling
Safety	"My knee hurts, what should I squat?"	A refusal	Safety
Explanation	"Why?" after a recommendation	An answer from stored evidence	Memory; no repeated lookup
Form question	"How do I do a hack squat properly?"	A labeled web answer with its link, or a notice that the lookup failed	Web rules and failure
Topic return	The user goes back to an earlier topic	A request to confirm the topic	Context switching

11. Details to Settle During the Build

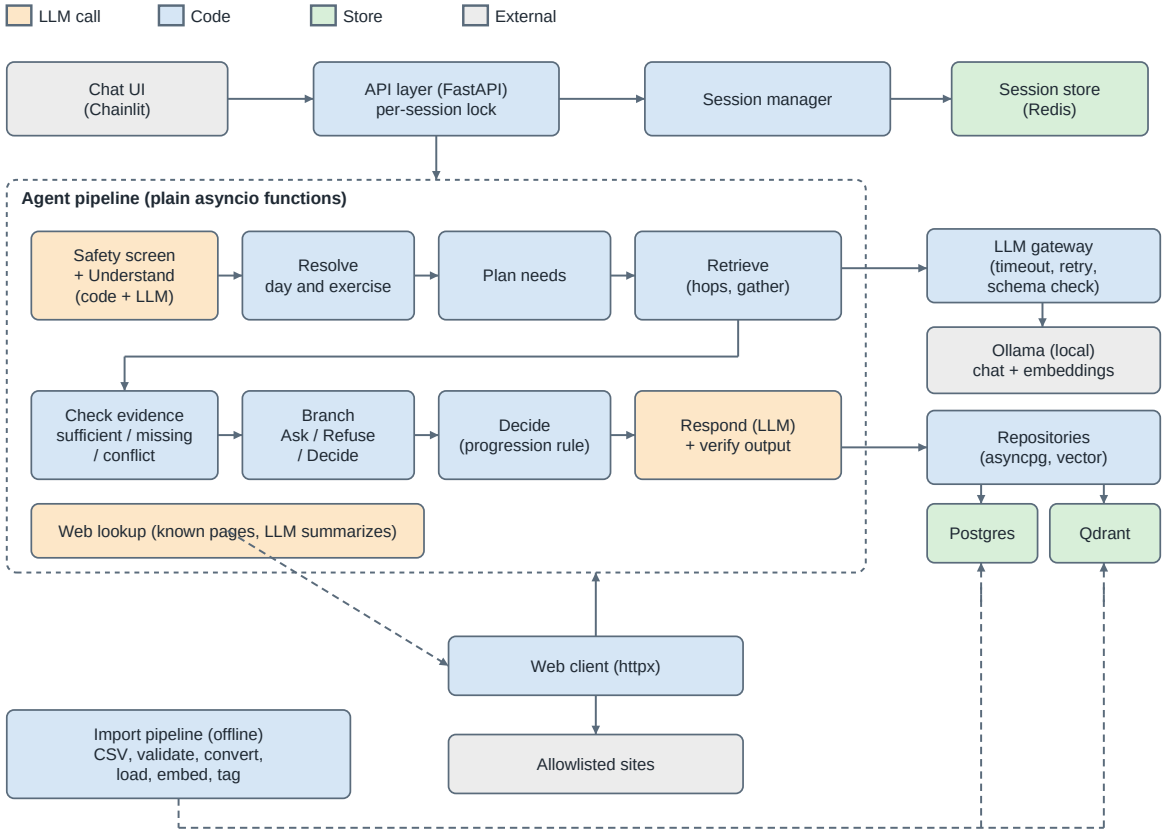
- How many repeated misses before the weight is reduced, and by how much.
- How recommendations are rounded to weights that exist (one option: weights seen in the user's own history).
- Check the chat model and the embedding model on small test sets of the user's own phrases before relying on them.
- Confirm how ACE and Jefit page addresses are built, by fetching about five samples, and review their terms.
- Tune the chat cap and the model concurrency limit by measurement.
- Check that Chainlit, mounted inside FastAPI, installs next to it without version conflicts and starts correctly. If not, run it as a separate process that calls the API.

Part 2: Design

12. Design Principles

- The language model handles language only: understanding the question and phrasing the answer. Facts, checks and calculations are code.
- Every fact comes from an SQL row or a labeled web source, and every recommendation carries an evidence record.
- Everything on the request path is asynchronous.
- Expected failures are typed results that lead to an answer, a question, a refusal or a system error, not surprise exceptions.
- All conversation state belongs to one chat and is only changed while holding that chat's lock.
- The pipeline is fixed: no step lets the model decide what happens next. This is deliberate: a small local model is unreliable at choosing steps, and the decisions here are safety-relevant.

13. Architecture



14. Components and Technology

Component	What it does	Technology
Chat UI	Sends messages and shows replies, web-source labels and evidence. Holds only the chat's session id; Chainlit's own login and chat storage are off.	Chainlit, mounted inside the FastAPI app
API layer	Receives the session id and message, takes the chat's lock, returns the reply.	FastAPI on uvicorn

Component	What it does	Technology
Session store	For each chat: capped messages and structured state (last recommendation with evidence, pending question, user-supplied values). Idle timeout by key expiry; the lock is a Redis lock with an expiry.	Redis through redis-py (async), behind a SessionStore interface
Agent pipeline	Fixed stages that end in an answer, a question, a refusal or a system error. One entry point, handle_message(session_id, text), is used by both the FastAPI route and the Chainlit handler.	Plain asyncio functions
Web lookup step	Builds the page address from the exercise name, fetches it, and has the model summarize form cues.	httpx; ACE and Jefit pages
Language model gateway	The only code that talks to the model: timeout, retry, schema validation and a concurrency limit.	Own wrapper over Ollama, using Ollama's schema parameter plus Pydantic validation and retry
Chat model	Understanding the question, phrasing the answer, summarizing web text.	granite4.1 8B on Ollama (llama3.1 8B as the comparison)
Repositories	Async data access that returns validated models. The rest of the code never sees SQL or vector calls.	asyncpg with plain SQL
PostgreSQL	Facts: exercises, plan, logs, sets, records, rules, exercise tags. Schema kept in numbered SQL files.	PostgreSQL
Vector store	Embeddings of exercise names and notes, for matching and search only.	Qdrant
Embedding model	Turns short text into vectors through Ollama's HTTP API.	all-minilm
Import pipeline	Offline: parses the Jefit CSV, validates, converts pre-2026 weights to kg, rounds, loads, embeds, matches each session to a plan day, and suggests exercise tags for review.	A Python script outside the request path
Settings and logging	Typed settings from environment variables; logs carry the session id.	pydantic-settings; standard logging

15. How a Request Flows

Standard request

- The UI passes the session id and the message to the pipeline entry point.
- The chat's lock is taken and its state is loaded. Messages in one chat queue; different chats run at the same time.
- If a question is pending, the reply is parsed first (simple parsing, then the model as a fallback), merged into the evidence, and the pipeline resumes at the evidence check, not from the start.
- Otherwise the pipeline runs: safety screen and understanding, resolving the day and exercise, working out what is needed, retrieval, the evidence check, then a question, a refusal or a decision, then the reply and its verification.
- The state is saved only after the request completes (history trimmed to the cap), so a failed request cannot corrupt it.
- The reply goes back with its labels and evidence identifiers, and the lock is released.

Weight for tomorrow. The day is proposed and confirmed. The exercise phrase "squat" is matched in Qdrant and in the planned exercises; several matches means a question. Then the last performance, the progression rule and the exercise tag are retrieved concurrently. The evidence check passes, the decision applies the rule, the reply is phrased, and verification checks every number against the evidence.

Missing data. No last performance is found, so the check reports a missing value. The pipeline stores a pending question with the partial evidence and asks the user. The next message resumes at the evidence

check.

"Why?" No retrieval. The stored evidence for the last recommendation is read from the session and phrased. If two recommendations are open, the assistant asks which one is meant.

16. Pipeline Stages

Stage	Kind	What it does	Can end with
Safety screen	Code	Pattern check of the raw text for acute pain, injury and diagnosis requests. Runs before the model and independently of it.	Refusal
Understand	Language model	Classifies the question and extracts the exercise phrase, date phrase and references to earlier answers into a validated structure. The model's own injury classification also triggers a refusal.	Refusal, question, system error
Resolve	Code and vector search	Proposes the day (least recently done, Arms excluded) and matches the exercise name.	Question
Plan needs	Code	Lists the values the question requires.	None
Retrieve	Code, async	Runs the lookups; independent ones together.	System error
Check evidence	Code	Returns sufficient, missing or conflicting.	Question, refusal
Decide	Code	Applies the progression rule using the exercise tag and builds the evidence record.	Refusal if still insufficient after asking
Respond	Language model	Phrases the answer from the evidence only.	System error, or a template reply
Verify	Code	Every number, date and exercise name in the reply must appear in the evidence, and web text must be labeled.	Template reply

17. Synchronous and Asynchronous Parts

Category	Items	Rule
Asynchronous	HTTP handlers; database queries; vector queries; model calls through Ollama; embeddings through Ollama; web fetches; session store access; waiting for a lock; timeouts.	Never block. Every call has a timeout.
Concurrent within a request	Last performance, progression rule, exercise tag and records, once the exercise is known.	Started together and collected together.
Throttled	Model calls, through a limit in the gateway.	Starts at 1 at a time; tuned by measurement on the 16 GB machine.
Synchronous, inline	Pydantic validation, evidence checks, progression arithmetic, unit conversion, reply verification, prompt assembly.	Pure CPU, microseconds: plain functions.
Offline, synchronous	The import pipeline.	A script, not part of the request path.

Where concurrency actually helps. Database and vector lookups take milliseconds, so running them together saves little. The real gains are that other chats keep being served while one waits on the model, the session store or a web page, and that web fetches never stall anything. A local model on one machine generates one or two answers at a time, so 100 simultaneous users queue at the model. The system stays responsive and correct, but their answers arrive one after another.

18. Where the Language Model Is Used

Call	Purpose	How its output is checked
Understand	Classify the question and extract entities.	Schema; retry on malformed output.
Reply parsing (fallback)	Read a clarification answer when simple parsing fails.	Schema and plausible range.
Web summary	Summarize the text of a fetched page into form cues.	Schema; labeled with the source.
Respond	Phrase the answer from the evidence.	Verified against the evidence.
Tag suggestion (offline)	Suggest compound, isolation or assisted for each exercise.	The user reviews and approves.

The model is not used in retrieval, the safety pattern screen, the evidence check, the decision, the calculation or session handling.

19. Where Data Is Retrieved

Source	What	Order
PostgreSQL	The plan day of each past session (worked out at import), planned exercises, last performance, set logs, progression rule, exercise tag, records.	Day first; then planned exercises; then last performance, rule and tag together.
Qdrant	Exercise name matching; notes.	Before the lookups that need the resolved exercise.
Web	Form and technique text from a page address built from the exercise name (ACE, Jefit).	Only for those questions; never in a calculation.

20. Retrieval Plans by Question Type

Each question type has a fixed plan. Steps in the same group run at the same time; each group waits for the one before it. Every plan ends with the evidence check, which decides between answering, asking and refusing.

Question type	Steps (groups run in order; items inside a group run together)	If something is missing or unclear
Weight or plan for tomorrow	1. Least recently done rotation day (Arms excluded), weekday as tie-break. 2. Planned exercises for that day. 3. For each exercise together: last session's sets, exercise tag, progression rule. 4. Code applies the rule set by set.	No history: ask which day. Tie: propose one and confirm. Exercise with no logs or no tag: no recommendation for that exercise, and say why.
Last time I did an exercise	1. Match the exercise name (Qdrant). 2. Latest session containing it, with its sets.	Several close matches: ask which one. No match: say it is not in her data. Never guess a near name.
Did I hit my targets	1. Match the exercise. 2. Planned target reps and the last session's sets together. 3. Code compares set by set.	No target set for the exercise: report what was done and say there was no target. Exercise not logged: say so.
Compare now vs. an earlier period	1. Match the exercise and turn the date phrase into two date ranges. 2. Sets for both ranges together. 3. Code computes best set, volume and the change.	A range has no sessions: say which one is empty. Vague phrase such as 'a while ago': ask for a range. Mixed units were already converted at import.

Question type	Steps (groups run in order; items inside a group run together)	If something is missing or unclear
Personal record	1. Match the exercise. 2. Records table and the best logged set together. 3. Code picks the higher value.	The two disagree: ask which to trust (conflicts are always asked). No record: say none is stored.
Form or technique	1. Match the exercise. 2. Fetch the known ACE and Jefit page for it. 3. Model summarizes the fetched text; the reply is labeled as web content.	Page missing or fetch fails: say the lookup failed and offer nothing invented. Pain or injury wording: refusal before any lookup.
Exercise outside her plan	1. Plan and her Jefit exercise catalog together. 2. Filter the catalog by muscle group and tag. 3. Optional web summary for the chosen exercise, labeled.	No untagged exercise is suggested. Muscle group unclear: ask. Never suggest an exercise that is not in her catalog.
Why did you say that	1. Read the stored evidence record for the last recommendation from the session. No other lookups.	Two recommendations open: ask which. Evidence expired with the session: say it is gone and offer to recompute.

Where the hops depend on each other: tomorrow's plan needs the day before the exercises, and the exercises before the last sets. Where they do not: the sets, tag and rule for different exercises all run at once, and the two date ranges in a comparison are fetched at once.

Open check for the build: every plan above should become a test that feeds a real database a missing or conflicting value and confirms the assistant asks or declines instead of answering.

21. Where Data Is Validated

Where	How	On failure
Import	Pydantic per row; units converted; weights rounded; the per-set table cross-checked against the logs text.	Bad rows are set aside with a report.
User input	Length and format; clarification replies range-checked and compared with history.	Ask again.
Model output	Schema enforced by Ollama, then Pydantic in the gateway.	Retry with the error as feedback, then fail with a typed error.
Retrieval results	Repositories return validated models.	System error.
Evidence	The check: sufficient, missing or conflicting.	Question or refusal.
Final reply	Every number and name traced to the evidence.	Template reply.
Web content	Approved sources only; treated as data; labeled.	Reported as unavailable.

22. Where State Lives

State	Where	How long
Facts (logs, plan, rules, tags)	PostgreSQL	Durable; rebuilt from the CSV by the import.
Embeddings	Qdrant	Rebuildable from PostgreSQL.
Messages and structured state	Redis, one key per chat	Until idle expiry or a new chat; survives a restart if persistence is on. A pending question keeps the key alive longer.
Configuration (approved sources, timeouts, limits)	Config files and environment	Static.

State	Where	How long
Request-scoped data	Local variables	One request.

There is no module-level mutable state. The per-chat lock is a Redis lock with an expiry, so a crashed request cannot hold a chat forever.

23. How Errors Move Through the System

Each stage returns either a value or a named failure. A router turns the failure into an outcome. Expected cases are not exceptions.

Failure	Handling	What the user sees
Database or vector store timeout or error	Retry reads once; then a system error. No guess, and no request for data as a workaround.	"I can't reach your workout data right now."
Language model timeout or unreachable	System error. For simple questions, such as listing a day's exercises, a template reply works without the model.	An error, or the template reply.
Malformed model output	Retry up to twice with the validation error; then a system error or a request to rephrase.	A request to rephrase.
Empty retrieval	Missing value: ask. Still missing after asking: refuse.	A question, then a refusal.
Conflicting records	Ask which one is right.	A question naming both values.
Web failure	Say so. Never fall back to the model's knowledge.	"The lookup failed."
Session store unreachable	System error: without the saved state the request cannot continue safely.	"I can't continue this chat right now."
Safety trigger	Refuse with fixed text; the model is bypassed.	A fixed message.
Unexpected exception	Caught at the API boundary and logged with the session id; the saved state is untouched.	A generic error.

Every call has its own timeout and the whole request has a deadline. Initial values, to be tuned: database 2 s, vector store 2 s, web 10 s, model 60 s, request 90 s. Concurrent lookups are collected so that one failure does not hide the others; the evidence check sees the partial results. Retries apply only to reads. If the client disconnects, the request's tasks are cancelled.

24. Technology Choices and Why

The reasons below are short drafts. Rewrite them in your own words before using them in an interview.

Layer	Choice	Why	Considered
Language model host	Ollama, local	Free, private, and runs on the laptop; the wrapper keeps it swappable.	Cloud API; llama.cpp; MLX
Chat model	granite4.1 8B	Fits about 8 to 9 GB of spare memory; Ollama lists tool use and structured JSON output.	llama3.1 8B, qwen2.5 7B, mistral 7B, gpt-oss 20B (too large)
Embeddings	all-minilm via Ollama	Tiny and fast; inputs are short English exercise names; called over HTTP so it stays async.	nomic-embed-text, embeddinggemma, mxbai-embed-large
Relational database	PostgreSQL with asyncpg	Native async driver, real network timeouts to test, production-style.	SQLite with aiosqlite, DuckDB
Vector store	Qdrant	Native async client, a separate failure mode to test, widely used in production.	pgvector, LanceDB, Chroma
API framework	FastAPI	Async, Pydantic-native, largest ecosystem.	Litestar, Starlette, aiohttp

Layer	Choice	Why	Considered
Structured output	Ollama schema parameter plus own Pydantic validation and retry	Teaches the full mechanism; no extra dependency; one place that knows how the model is called.	Instructor, PydanticAI, Outlines
Session store	Redis (async)	Built-in expiry for the idle timeout; state survives a restart; a new tool to learn.	In-memory dict, Postgres table
HTTP client	httpx	Async, simple, granular timeouts, also the test client for FastAPI.	aiohttp
Finding web pages	Direct fetch of known pages	No account or search service; fewer failure points.	A search API; pre-fetching at import
Chat UI	Chainlit	Chat-first, async, quick to build.	Plain HTML and JavaScript, Gradio, Streamlit
Test runner	pytest with pytest-asyncio	The common default.	The anyio plugin
HTTP mocking	pytest-httpx	Mocks httpx directly.	respx
Services in tests	Real Postgres, Qdrant and Redis in containers	Real timeouts and behaviour.	Fakes
Concurrency demo	A plain asyncio script with httpx	Small, and shows interleaving and isolation directly.	Locust
Failure injection	In-process: a fake model, forced timeouts	Covers most of the required failures cheaply.	A network fault proxy
Packages	uv	Fast, with a lockfile.	Poetry, pip with venv
Lint and format	ruff	One fast tool for both.	black with flake8
Type checking	pyright	Fast.	mypy
Containers	Docker Desktop and Compose	Runs Postgres, Qdrant and Redis; Ollama stays on the Mac because containers cannot use its GPU.	Colima, OrbStack
Migrations	Plain numbered SQL files	Simple; the database can be rebuilt by the import.	Alembic
Settings and logging	pydantic-settings, standard logging	Typed environment settings; no extra dependency.	structlog

25. Testing Approach

- Write tests before the code, then run the whole suite and confirm it passes.
- Unit tests cover pure logic: progression, evidence checks, unit conversion, day matching, reply verification.
- Integration tests run against real PostgreSQL, Qdrant and Redis containers.
- Failure tests use a fake model and forced timeouts for: model timeout, database timeout, vector store failure, web failure, malformed output, empty retrieval and conflicting records.
- A synthetic CSV supplies conflict and missing-data cases that the real export cannot.
- A set of about 30 sample messages with expected structured output validates the chat model, and a set of messy exercise phrases validates the embeddings.
- A concurrency script runs many chats at once and checks that they never mix state and that nothing blocks.

26. Folder Structure

```
workout-agent/
|-- pyproject.toml        uv, ruff and pyright settings
```

```
|-- uv.lock
|-- docker-compose.yml    postgres, qdrant, redis (Ollama runs on the host)
|-- .env.example          settings template
|-- .gitignore            data/, .env
|-- data/                 Jefit export (git-ignored)
|-- config/               progression_rules.yaml, sources.yaml, exercise_tags.yaml
|-- migrations/           0001_*.sql, 0002_*.sql ...
|-- scripts/              import_jefit, apply_migrations, embed_exercises,
|                         suggest_tags, concurrency_demo
|-- src/workout_agent/
|   |-- settings.py, logging.py
|   |-- schemas/          Pydantic: rows, intent, evidence, session state
|   |-- llm/              gateway, Ollama client, fake model for tests
|   |-- embeddings/       all-minilm client
|   |-- repositories/     plan, history, rules, tags, vectors, web pages
|   |-- session/          SessionStore interface, Redis store, lock
|   |-- importing/        Jefit parser, unit normalization, day matching
|   |-- agent/            safety, understand, resolve, plan_needs, retrieve,
|   |                     check_evidence, decide, respond, verify, web_lookup,
|   |                     pipeline, handle_message
|   |-- api/              FastAPI app and routes
|   `-- ui/               Chainlit app
`-- tests/
    |-- unit/  integration/ (real containers)  failure/  concurrency/
    `-- fixtures/         synthetic CSV for conflict and missing-data cases
```

27. Build Order

Phase	What gets built	Done when
1. Foundation	uv project, Compose file, schemas, migrations.	Containers start; schemas validate.
2. Import	Split the Jefit CSV, validate, convert units, load PostgreSQL, match sessions to plan days.	The real export loads; bad rows are reported.
3. Retrieval	Repositories, concurrent lookups, the evidence check, with an in-memory session store behind the interface.	Weight lookup returns structured evidence; missing and conflict cases are detected.
4. Decisions	Exercise tags and the recommendation engine.	Recommendations match the progression rule on test cases.
5. Language model	The gateway, understanding, replying and verification.	The sample-message set passes; failure tests pass.
6. Sessions	Redis session store, locks, asking and resuming.	A pending question survives the next message and a restart.
7. Web	Page fetching for form questions, labeling and failure handling.	Form answers are labeled; failures are reported.
8. Interface	FastAPI routes and the Chainlit chat.	All journeys work in the browser.
9. Reliability	Concurrency script, timing measurements, tuning of limits.	Many chats run at once without mixing state.